WEEK 12: APPROXIMATION ALGORITHMS

Advanced Algorithms

BREAKING NEWS

The Exponential Time Hypothesis is false

Proof released by OpenAI on October 25, 2026

ETH IS FALSE?

ETH (Week 2) There is a constant $c > 0$ such that solving 3-SAT on n variables requires time $\Omega(2^{cn})$.

Previous best: about 1.3^n (PPSZ and refinements).

Theorem (OpenAI, October 2026) A deterministic algorithm *counts* the satisfying assignments of any 3-CNF formula in time $\mathrm{poly}(n, m) \cdot 2^{O(n \log\log n / \sqrt{\log n})}$.

Preprint, October 25, 2026; not yet peer reviewed.

3-SAT AS A POLYNOMIAL

The clause $x_a \vee \neg x_b \vee x_c$ fails only at $x_a = 0,\ x_b = 1,\ x_c = 0$. So

$$\Phi(X_a, X_b, X_c) \ = \ 1 - (1 - X_a)\, X_b\, (1 - X_c)$$

is 1 on satisfying inputs and 0 otherwise.

In general: $\Phi_e = 1 - \prod_{L \in e}(1 - L)$, where the literal x_v becomes X_v and $\neg x_v$ becomes $1 - X_v$.

COUNTING WITH A POLYNOMIAL

$$\#F = \sum_{x \in \{0,1\}^n} \prod_{e \in F} \Phi_e(x)$$

- Each term is 1 if x satisfies every clause, and 0 otherwise.
- Assume each variable appears in at most 3 clauses (the general case reduces to this). Then the product has degree $\leq 3n$.
- Evaluating the sum one x at a time is brute force: 2^n evaluations.

WHEN THE SUM IS EASY

Suppose the variables split into groups $S_1, \ldots, S_r$ of size n/r, and every clause lies inside one group.

- Then $\#F = N_1 N_2 \cdots N_r$, where N_g counts the satisfying assignments of group g's clauses.
- Cost: $r \cdot 2^{n/r}$ instead of 2^n.

Real formulas don't split. **Plan:** build a polynomial $H(T)$ with $H(0) = \#F$ that is cheap to evaluate at $T = 1, \ldots, Q$. Then recover $H(0)$ from those values.

LINEAR REPLACEMENT

In clause e, replace each input X_v by a **linear form** in all the variables, with coefficients that are polynomials in T:

$$X_v \;\rightsquigarrow\; \ell_{e,v}(T) \;=\; \sum_{i=1}^{n} p_{e,v,i}(T)\, X_i .$$

- At $T = 0$ nothing changes: $\ell_{e,v}(0) = X_v$, i.e. $p_{e,v,v}(0) = 1$ and $p_{e,v,i}(0) = 0$ for $i \neq v$.
- Each coefficient $p_{e,v,i}$ has degree $\leq R$ in T.
- Each occurrence gets its own form: x_v may be replaced differently in different clauses.

A POLYNOMIAL IN T

$$H(T) = \sum_{x \in \{0,1\}^n} \prod_{e \in F} \Phi_e\left(\ell_{e,v}(T)\right)_{v \in e}$$

- $H(0) = \#F$: at $T = 0$ every form is the original variable.
- H is a polynomial in one variable T. Its degree is at most $3n \cdot R$, since the inputs enter with degree $\leq 3n$ and each has degree $\leq R$ in T.
- At $t \neq 0$ the inputs need not be 0 or 1, so $H(t)$ counts nothing. It is just a number, and we only need it to be cheap.

CHEAP POINTS $T = 1, \ldots, Q$

The forms are built so that at each $t \in \{1, \ldots, Q\}$:

- the variables split into r groups; each clause is assigned to one;
- a clause reads its own group freely, but other groups only through a few shared **measurements** (small integer linear combinations);
- fixing the measurement values splits the sum into independent group sums, as on the “easy” slide.

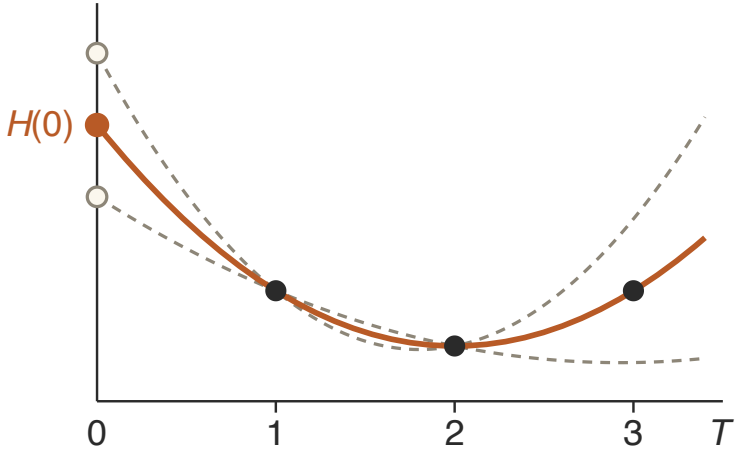
Cost per point: (number of measurement values) $\times\ 2^{O(n/r)} = 2^{o(n)}$.

THE DEGREE BUDGET

- Suppose a term of $\ell_{e,v}$ reaches outside clause e's group only at $t = 2$. Multiply it by $(1 - T/2)$: unchanged at $T = 0$, gone at $T = 2$. Cost: one degree.
- The lazy fix, $\ell_{e,v}(T) = \prod_{t=1}^{Q}(1 - T/t)\, X_v$, vanishes at every cheap point — but has degree Q, and we'll see that costs as much as brute force.
- The heart of the paper: forms of degree $R = \Theta(\sqrt{\log n})$ with $Q = \Theta(\log n / \log\log n)$ cheap points. So $R \ll Q$.

INTERPOLATION

FACT A polynomial of degree $\leq d$ is determined by its values at any $d+1$ points.



$d = 2$: many parabolas fit $H(1)$ and $H(2)$, each with its own $H(0)$.

Adding $H(3)$ leaves only $T^2 - 4T + 5$, so $H(0) = 5$.

Why: the difference of two such polynomials has degree $\leq d$ but $d+1$ roots. A nonzero polynomial of degree $\leq d$ has at most d roots, so the difference is 0.

INTERPOLATION IN T

We want $H(0)$, but H is cheap only at $T = 1, \ldots, Q$.

- If $Q > d = 3nR$: compute $H(1), \ldots, H(Q)$, solve Q linear equations for the coefficients of H, and read off $H(0) = \#F$.
- But $Q = \Theta(\log n / \log \log n) \ll 3nR$.

ONE POINT, MANY COEFFICIENTS

The **Taylor coefficients** $c_k = H^{(k)}(t)/k!$ of H at a point t:

$$H(t+z) \;=\; c_0 + c_1 z + c_2 z^2 + \cdots + c_d z^d.$$

- They determine H: substitute $z = T - t$. In particular, $z = -t$ gives $H(0)$.
- Example: $H(T) = T^2 - 4T + 5$ at $t = 1$ gives $H(1+z) = 2 - 2z + z^2$, so $H(0) = 2 + 2 + 1 = 5$.

So one cheap point would do, given $3nR + 1$ Taylor coefficients there. But each extra coefficient costs time.

FEW POINTS, MORE DATA

Combine the two: at each of the Q points, compute the first J Taylor coefficients of H.

> **CLAIM** If $QJ > 3nR$, this data determines H.

Proof. Two polynomials of degree $\leq 3nR$ with the same data differ by a multiple of $(T-t)^J$ for every t, hence of $\prod_{t=1}^{Q}(T-t)^J$, which has degree $QJ > 3nR$. So they are equal. ■

More Taylor coefficients cost more time. The total stays $2^{o(n)}$ when $J \approx 3nR/Q = o(n)$ — that is, when $R \ll Q$.

WHAT IT MEANS

- If the proof holds, ETH and SETH are false. Week 2's SETH lower bound for LCS and Week 11's "no $2^{o(n)}$ Subset Sum under ETH" lose their foundation.
- It does *not* make every NP-complete problem $2^{o(N)}$: a reduction to 3-SAT with N^2 variables gives only $2^{o(N^2)}$.
- It does count independent sets and q-colorings of a graph on n vertices in $2^{o(n)}$ time.

PART 1: WHY APPROXIMATE?

WHERE WEEK 11 LEFT US

Vertex cover, independent set, 3SAT, Hamiltonian path … are **NP-complete**. No polynomial-time exact algorithm unless P = NP.

But the inputs keep coming. What are our options?

- **Exact, exponential:** become infeasible on large enough inputs.
- **Special structure:** vertex cover is easy on trees (W3) and bipartite graphs (W8) — but our graph may be neither.

WHERE WEEK 11 LEFT US

- **Heuristics:** often good, but *no guarantee* — we can't tell a good answer from a terrible one.
- **Approximation algorithms:** polynomial time, *and* a provable worst-case guarantee on solution quality.

Today: approximation algorithms. We trade exactness for speed — but keep a *proof*.

APPROXIMATELY OPTIMAL

Definition For a minimization problem, an algorithm is an **α-approximation** ($\alpha \geq 1$) if it runs in polynomial time and, on *every* input, outputs a feasible solution of cost $\mathrm{ALG} \leq \alpha \cdot \mathrm{OPT}$.

- For maximization: $\mathrm{ALG} \geq \mathrm{OPT}/\alpha$ (so $\alpha \geq 1$ in both conventions).
- **Worst case, per instance** — not on average, not "usually". A 2-approximation for vertex cover never returns a cover more than twice the smallest.
- Smaller α is better; $\alpha = 1$ means exact.

THE ONE TRICK BEHIND EVERY PROOF TODAY

To prove $\mathrm{ALG} \leq \alpha \cdot \mathrm{OPT}$ we would seem to need OPT — the very number that is NP-hard to compute.

KEY IDEA Compare against a **lower bound** instead. Find an efficiently analyzable quantity $\mathrm{LB} \leq \mathrm{OPT}$ and show $\mathrm{ALG} \leq \alpha \cdot \mathrm{LB}$. Then

$$\mathrm{ALG} \;\leq\; \alpha \cdot \mathrm{LB} \;\leq\; \alpha \cdot \mathrm{OPT}.$$

THE ONE TRICK

Every algorithm today is really a *pair*: a solution, and a lower bound certifying its quality.

Problem	Lower bound on OPT	Source
Vertex cover	size of any matching	W3
Vertex cover (weighted)	LP optimum / any dual solution	W9–10

PART 2: VERTEX COVER VIA MAXIMAL MATCHING

RECALL: COVERS AND MATCHINGS

Definition A **vertex cover** of $G = (V, E)$ is a set $C \subseteq V$ touching every edge: for all $\{u, v\} \in E, u \in C$ or $v \in C$.

Definition A matching M is **maximal** if no edge of G can be added to it: every edge of G shares an endpoint with some edge of M.

RECALL: COVERS AND MATCHINGS

CAREFUL Maximal $\neq$ maximum. A maximal matching is merely *unextendable*; a maximum matching has the largest possible size. Greedy (W4 flavour) gives a maximal matching in $O(m)$ time — no augmenting paths, no flows needed.

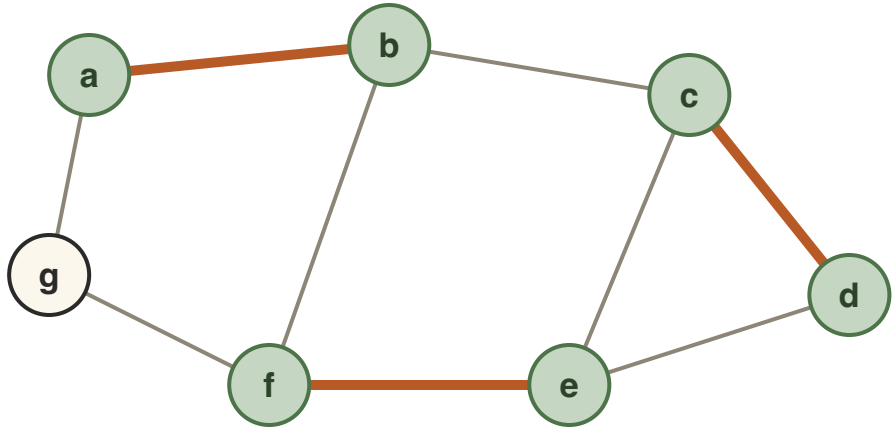
THE ALGORITHM (GAVRIL / YANNAKAKIS)

```
function MATCHING-VC(G = (V, E)):
    M ← ∅
    for each edge {u, v} ∈ E:              // any order
        if u unmatched and v unmatched:
            add {u, v} to M                // M stays a matching
    return C = { every endpoint of every edge in M }
```

- One pass over the edges: $O(m)$ time.
- M is maximal by construction: any skipped edge had a matched endpoint at the time — and vertices never become unmatched.
- Output size: $|C| = 2|M|$.

Two claims to prove: C is a cover, and $|C| \leq 2\,\mathrm{OPT}$.

WORKED EXAMPLE



Maximal matching $M = \{ab, cd, ef\}$ (orange); the cover is all matched endpoints (green). $|M| = 3$, output $|C| = 6$; here $\mathrm{OPT} = 4$ (e.g. $\{b, c, e, g\}$), so the ratio is $1.5 \leq 2$.

CORRECTNESS: C COVERS EVERY EDGE

- Suppose some edge $\{u, v\}$ has *neither* endpoint in C.
- Then neither u nor v is an endpoint of any matching edge — both are unmatched.
- So $M \cup \{\{u, v\}\}$ is still a matching — contradicting maximality of M.
- Hence every edge is covered: C is a vertex cover. ■

THE FACTOR OF 2

- The edges of M are pairwise disjoint (it's a matching).
- Any vertex cover — including an optimal one — must cover each edge of M, and *no single vertex covers two of them*.
- So every vertex cover uses $\geq |M|$ distinct vertices: $\mathrm{OPT} \geq |M|$.
- Therefore

$$|C| \;=\; 2|M| \;\leq\; 2\,\mathrm{OPT}.$$

THE FACTOR OF 2

THEOREM MATCHING-VC is a 2-approximation for (unweighted) vertex cover, in $O(m)$ time.

REMARKS ON THE MATCHING ALGORITHM

- **The analysis is tight.** On $K_{n,n}$ the algorithm finds a perfect matching and outputs all $2n$ vertices, while one side (n vertices) is an optimal cover. Ratio $\to 2$.
- **Simplicity is a feature:** no LP, no flow, one pass.

REMARKS ON THE MATCHING ALGORITHM

- **Limitations:** it ignores vertex weights — both endpoints go in, however expensive. Weighted VC needs the next two methods.

State of the art: $2 - \Theta(1/\sqrt{\log n})$ (Karakostas 2005). Beating $2 - \varepsilon$ is NP-hard (Khot–Regev 2008, given the 2026 OpenAI proof of the Unique Games Conjecture). So factor 2 is essentially the best possible.

PART 3: VERTEX COVER VIA LP ROUNDING

LP ROUNDING (RECALL WEEK 9)

Weighted vertex cover, relaxed: $\min \sum_v w_v x_v$ s.t. $x_u + x_v \geq 1$ on every edge, $x \geq 0$.

```
function LP-ROUND-VC(G, w):
    solve the LP relaxation  →  optimal x*      // poly time (W9)
    return C = { v : x*_v ≥ 1/2 }               // round halves up
```

WEEK 9 PROVED C is a cover (every edge has an endpoint with $x^* \geq \frac{1}{2}$), and $w(C) \leq 2\,\mathrm{OPT}_{LP}$ (each $v \in C$ has $2x^*_v \geq 1$).

LP ROUNDING AS AN ALGORITHM

THEOREM LP rounding is a 2-approximation for **weighted** vertex cover: $w(C) \leq 2\,\mathrm{OPT}_{LP} \leq 2\,\mathrm{OPT}$.

- Week 9 read this as a fact about the LP: its integrality gap is at most 2. Today it is an *algorithm* — polynomial time, with a guarantee.

LP ROUNDING AS AN ALGORITHM

- It even certifies itself: OPT_{LP} is a lower bound computed along the way, so $w(C)/\mathrm{OPT}_{LP}$ bounds this run's actual ratio.
- Same factor as the matching algorithm, but now with weights: the LP weighs each vertex's cost when choosing which endpoints to cover.

WHY 2 IS A WALL

- Week 9: on K_n the LP takes every $x_v = \frac{1}{2}$, value $\frac{n}{2}$, while any cover needs $n - 1$ vertices. The gap is $2 - \frac{2}{n}$.
- Any proof of the form $\mathrm{ALG} \le \alpha \, \mathrm{OPT}_{LP}$ forces $\alpha \ge$ integrality gap. So *this* lower bound cannot certify better than ≈ 2.
- And not just for LPs: beating $2 - \varepsilon$ is NP-hard (Khot–Regev), so *no* polynomial-time algorithm gets past 2 unless P = NP.

PART 4: VERTEX COVER, PRIMAL–DUAL

CAN WE HAVE BOTH?

- **Maximal matching:** no LP — but ignores weights.
- **LP rounding:** handles weights — but solves an LP first.

Both analyses charge the cover to a **lower bound** on OPT: a matching, or OPT_{LP}. Only the second needs a solver.

Week 10 offers cheaper lower bounds: *any* feasible dual solution. Plan: build a dual solution greedily, and pay for the cover out of it.

THE DUAL: PACKING EDGES (W10 RECIPE)

Applying the dualization recipe from Week 10 to the VC relaxation (one dual variable $y_e \geq 0$ per edge constraint):

Primal (fractional cover)

$$\begin{aligned} \min \ & \textstyle\sum_v w_v x_v \\ \text{s.t. } & x_u + x_v \geq 1 \quad \forall \{u, v\} \\ & x \geq 0 \end{aligned}$$

Dual (fractional edge packing)

$$\begin{aligned} \max \ & \textstyle\sum_e y_e \\ \text{s.t. } & \textstyle\sum_{e \ni v} y_e \leq w_v \quad \forall v \\ & y \geq 0 \end{aligned}$$

THE DUAL: PACKING EDGES (W10 RECIPE)

- Interpretation: put money y_e on each edge, never loading any vertex beyond its price w_v — a *fractional matching*. (Bipartite + unit weights: exactly the Kőnig matching/cover pair of W10.)
- **Weak duality (W10):** every feasible y satisfies $\sum_e y_e \le \mathrm{OPT}_{LP} \le \mathrm{OPT}$. Any feasible dual is a lower bound — no need to solve either LP to optimality!

THE PRIMAL–DUAL IDEA

Complementary slackness (W10) says: at optimality, $x_v > 0 \;\Rightarrow\; \sum_{e \ni v} y_e = w_v$ — you only buy vertices whose dual constraint is **tight**.

Primal–dual algorithms take this as a *design rule* rather than an optimality condition.

THE PRIMAL–DUAL IDEA

The design rule, in three steps:

- Grow a feasible dual y from 0 (our lower bound grows with it).
- Only add vertex v to the cover when its constraint becomes tight: $\sum_{e \ni v} y_e = w_v$. Its cost is then *paid for* by the dual money on its edges.
- Keep raising duals on edges that are still uncovered, until none remain.

PRIMAL–DUAL ALGORITHM

```
function PRIMAL-DUAL-VC(G, w):
    y_e ← 0 for all e;   C ← ∅
    slack(v) ← w_v for all v          // slack(v) = w_v − Σ_{e∋v} y_e
    while some edge {u, v} has u ∉ C and v ∉ C:
        δ ← min(slack(u), slack(v))
        y_uv ← y_uv + δ                  // raise until an endpoint is tight
        slack(u) ← slack(u) − δ;  slack(v) ← slack(v) − δ
        add every endpoint with slack 0 to C   // tight ⇒ bought
    return C
```

- y stays dual-feasible: we never raise y_{uv} past the smaller remaining slack.
- Each iteration covers its edge (one endpoint goes tight), so at most m iterations: $O(m)$ arithmetic. No LP solver anywhere.
- Termination condition *is* feasibility: the loop exits only when every edge has an endpoint in C.

PRIMAL–DUAL: THE FACTOR OF 2

- Every $v \in C$ was added when tight: $w_v = \sum_{e \ni v} y_e$.
- Sum over the cover:

$$w(C) \;=\; \sum_{v \in C} \sum_{e \ni v} y_e \;\le\; 2 \sum_{e \in E} y_e,$$

  since each edge e has only 2 endpoints, so y_e is counted at most twice.
- y is dual-feasible, so by weak duality $\sum_e y_e \le \mathrm{OPT}_{LP} \le \mathrm{OPT}$.
- Chain them: $w(C) \le 2 \sum_e y_e \le 2\,\mathrm{OPT}$. ■

PRIMAL–DUAL: THE FACTOR OF 2

Theorem PRIMAL-DUAL-VC is a 2-approximation for weighted vertex cover in $O(m)$ time — LP-quality guarantee, no LP solved.

THREE 2-APPROXIMATIONS

Algorithm	Weights?	Solves an LP?	Lower bound used
Maximal matching	no	no	$\lvert M\rvert \le \mathrm{OPT}$
LP rounding	yes	yes	$\mathrm{OPT}_{LP} \le \mathrm{OPT}$
Primal–dual	yes	no	$\sum_e y_e \le \mathrm{OPT}_{LP} \le \mathrm{OPT}$

THREE 2-APPROXIMATIONS, ONE PROBLEM

- Cute fact: with unit weights, primal–dual *is* the matching algorithm. Raising y_{uv} to 1 makes both endpoints tight at once, so tight vertices = matched endpoints, and the raised edges form a maximal matching.
- Same factor three ways is no accident: all three proofs bound the output against (something at most) the LP optimum, and the integrality gap of that LP is $2 - o(1)$.

PART 5: METRIC TSP VIA MST

THE TRAVELLING SALESPERSON PROBLEM

Problem Given costs $c(u, v)$ on all pairs from a set of n cities, find a minimum-cost **tour**: a cycle visiting every city exactly once.

TSP is NP-hard (its decision version contains Hamiltonian cycle, W11). Worse: *approximating* general TSP is hopeless.

THE TRAVELLING SALESPERSON PROBLEM

CAREFUL For arbitrary costs, no α-approximation exists for any constant α unless P = NP. Given a graph G, set $c(u, v) = 1$ on edges of G and $c(u, v) = \alpha n + 1$ off them: an α-approximation returns a tour of cost $\leq \alpha n$ iff G has a Hamiltonian cycle — deciding an NP-complete problem.

So we need an honest extra assumption on the costs.

METRIC TSP

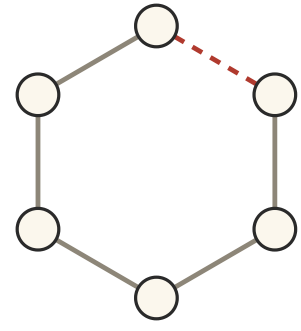
ASSUMPTION The costs form a **metric**: $c(u, v) = c(v, u) \geq 0$ and the **triangle inequality** holds: $c(u, w) \leq c(u, v) + c(v, w)$ for all u, v, w.

- "Direct is never worse than via a detour." Points in the plane, road networks, shortest-path distances in any graph: all metrics.
- Metric TSP is still NP-hard — but now approximable.
- Our lower bound will come from an old friend: the **minimum spanning tree** (Weeks 5–6).

THE LOWER BOUND: $\mathrm{MST} \le \mathrm{OPT}$

- Take an optimal tour, cost OPT.
- Delete any one edge of it.
- What's left is a spanning *path* — in particular a spanning tree.
- Its cost is $\le \mathrm{OPT}$ (we removed a nonnegative-cost edge), and the MST costs no more:

$$c(\mathrm{MST}) \;\le\; \mathrm{OPT}.$$



A tour minus one edge (dashed red) is a spanning tree.

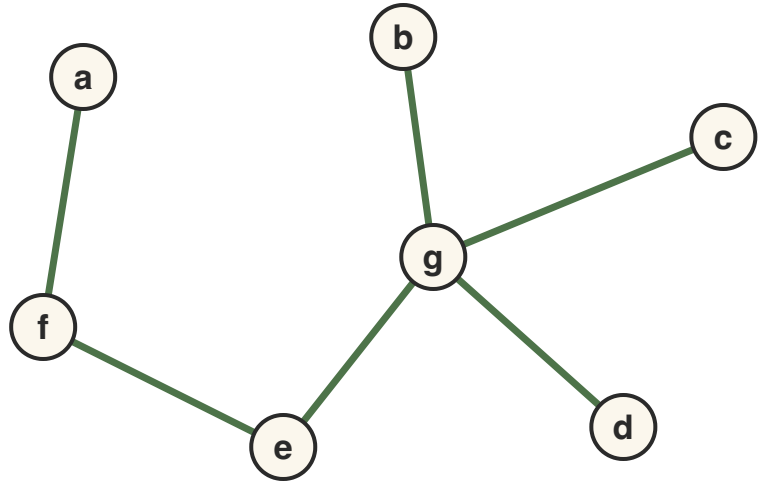
A tree we can compute exactly in $O(m \log n)$ (Kruskal, Prim) lower-bounds a quantity that is NP-hard to compute. That asymmetry powers the algorithm.

THE DOUBLE-TREE ALGORITHM

```
function DOUBLE-TREE-TSP(cities, metric c):
    T ← MST of the complete graph on the cities   // Kruskal / Prim, W5–6
    D ← T with every edge doubled                 // every degree is now even
    W ← Euler tour of D                           // connected + even degrees ⇒ exists
    H ← shortcut W: follow W, skip any city already visited
    return H                                       // a cycle through every city once
```

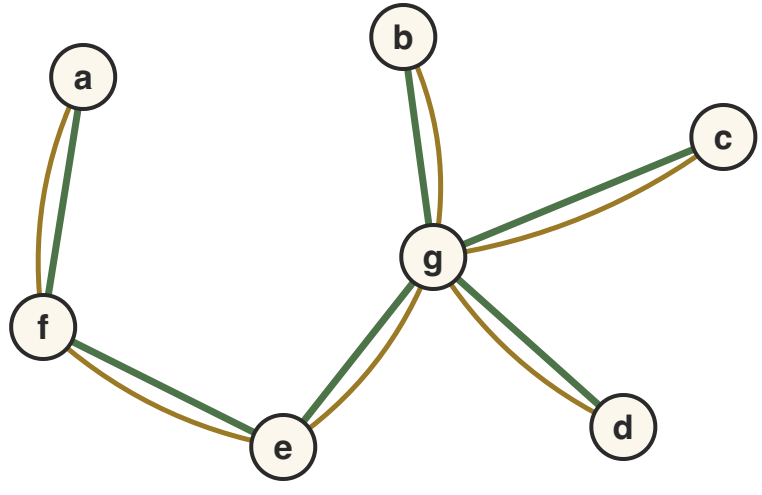
- **Euler tour:** a closed walk using every edge of D exactly once — it exists precisely because doubling made all degrees even, and it visits every city (possibly several times).
- **Shortcutting:** whenever the walk would revisit a city, jump directly to the next new one — legal because the graph is complete, and *cheap* because of the triangle inequality.

STEP 1: THE CITIES AND THEIR MST



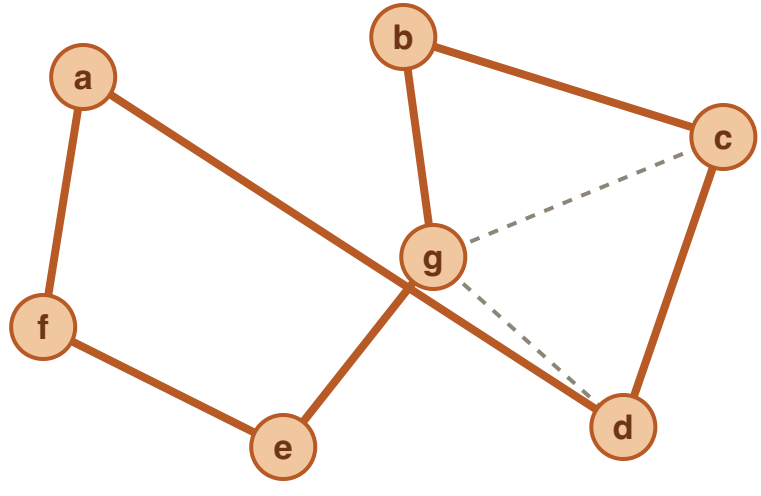
Seven cities in the plane (Euclidean metric); their MST in green: $c(\mathrm{MST}) \leq \mathrm{OPT}$.

STEP 2: DOUBLE EVERY EDGE



Second copies (gold, curved). All degrees are even, so an Euler tour exists: $a\ f\ e\ g\ b\ g\ c\ g\ d\ g\ e\ f\ a$, of total cost $2 \cdot c(\mathrm{MST})$.

STEP 3: SHORTCUT THE EULER TOUR



Euler walk $a\ f\ e\ g\ b\ \underline{g}\ c\ \underline{g}\ d\ \underline{g\ e\ f}\ a$; skipping revisits (underlined) gives the tour $a\ f\ e\ g\ b\ c\ d\ a$ (orange). Each skip is a triangle-inequality shortcut.

METRIC TSP: THE FACTOR OF 2

- The Euler tour W uses each edge of the doubled tree once:

$$c(W) \;=\; 2\,c(\mathrm{MST}).$$

- Shortcutting replaces a stretch $u \to x_1 \to \cdots \to x_k \to v$ of W by the single edge uv; by (repeated) triangle inequality this never increases the cost:

$$c(H) \;\le\; c(W).$$

METRIC TSP: THE FACTOR OF 2

- And the MST is our lower bound: $c(\mathrm{MST}) \le \mathrm{OPT}$.
- Chain: $c(H) \le 2\,c(\mathrm{MST}) \le 2\,\mathrm{OPT}$. ■

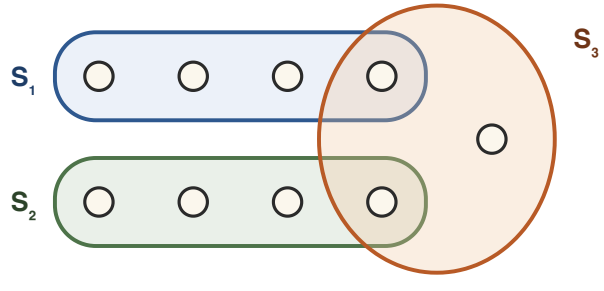
THEOREM Double-tree is a 2-approximation for metric TSP.

Can do better: replace "double the tree" by "add a min-cost perfect matching on the odd-degree vertices" — Christofides (1976) gets factor $\frac{3}{2}$; only in 2021 was $\frac{3}{2} - 10^{-36}$ achieved (Karlin–Klein–Oveis Gharan).

PART 6: GREEDY SET COVER

SET COVER

PROBLEM Given a universe U of n elements and sets $S_1, \ldots, S_m \subseteq U$ with $\bigcup_i S_i = U$, choose the fewest sets whose union is U.



Nine elements, three sets; here all three sets are needed.

SET COVER

- Vertex cover is the special case: elements = edges, one set per vertex (its incident edges) — each element in exactly 2 sets. So set cover is NP-hard too.
- Real face of the problem: choosing test suites, sensor placement, crew scheduling, feature selection …

THE GREEDY ALGORITHM

```
function GREEDY-SET-COVER(U, S_1..S_m):
    R ← U                                  // still uncovered
    picked ← ∅
    while R ≠ ∅:
        S ← a set maximizing |S ∩ R|     // covers the most uncovered
        picked ← picked ∪ {S};   R ← R \ S
    return picked
```

The obvious algorithm — grab whatever helps most right now. Runs in polynomial time. The question is how much it can lose.

THE GREEDY ALGORITHM

THEOREM Greedy uses at most $H_n \cdot \mathrm{OPT}$ sets, where $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n} \leq \ln n + 1.$

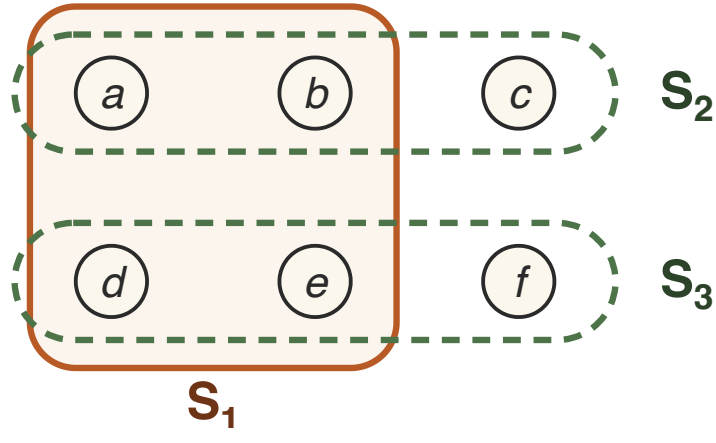
A $(\ln n + 1)$-approximation — our first non-constant factor. We prove it by *charging* the cost to elements.

CHARGING, STEP 1: EVERY ELEMENT PAYS A PRICE

- When greedy picks a set S that covers t *new* elements, spread its cost over them: each of the t elements is charged $\text{price}(e) = 1/t$.
- Every picked set distributes exactly 1 unit, and every element is charged exactly once (when first covered), so

$$\#\{\text{sets greedy picks}\} \;=\; \sum_{e \in U} \text{price}(e).$$

EXAMPLE: PRICES



- Greedy picks S_1 first: 4 new elements (each row has 3). So a, b, d, e pay $\frac{1}{4}$ each.
- Then S_2: only c is new (a and b have already paid), so c pays 1. Then S_3: f pays 1.
- Total: $4 \cdot \frac{1}{4} + 1 + 1 = 3$, the number of sets greedy picked. But $\mathrm{OPT} = 2$: the two rows S_2, S_3.

CHARGING, STEP 1: EVERY ELEMENT PAYS A PRICE

Plan: bound each element's price, then sum.

Prices are highest late in the run, when picked sets cover only a few new elements each.

CHARGING, STEP 2: THE KEY LEMMA

LEMMA Consider the moment when k elements are still uncovered. Then every element covered by greedy's next pick is charged at most OPT/k.

- The optimal solution's OPT sets cover all of U — in particular all k remaining elements.
- By pigeonhole, some single set covers $\geq k/\mathrm{OPT}$ of them.

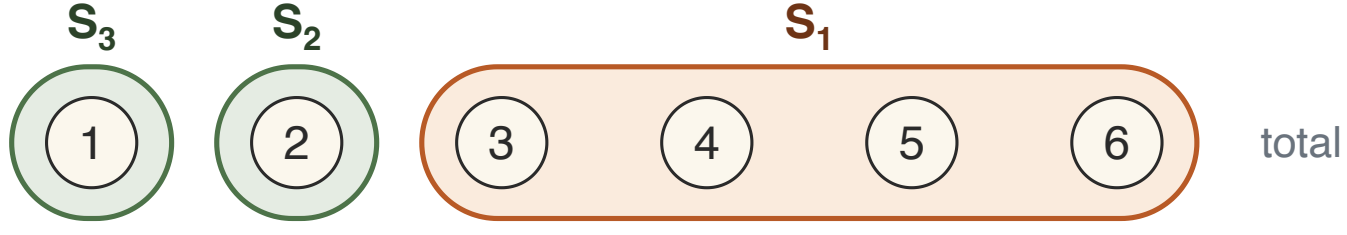
  (This is today's lower-bound trick, inverted: OPT's mere *existence* promises a good set.)

CHARGING, STEP 2: THE KEY LEMMA

- Greedy picks a set covering the *most* uncovered elements, so its pick covers $t \geq k/\mathrm{OPT}$ new elements.
- Each of them pays $1/t \leq \mathrm{OPT}/k$. ■

EXAMPLE: BOUNDING THE PRICES

Number the elements by when greedy covers them, last first. Each set is drawn over the elements it newly covers. Here $\mathrm{OPT} = 2$.



price	1	1	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	3
	≤	≤	≤	≤	≤	≤	≤
OPT/k	2	1	$\frac{2}{3}$	$\frac{1}{2}$	$\frac{2}{5}$	$\frac{1}{3}$	4.9

The slack: elements 3–6 were all covered when 6 remained, so each pays at most $2/6$. Yet OPT/k charges element k as if only k remained.

CHARGING, STEP 3: SUM THE HARMONIC SERIES

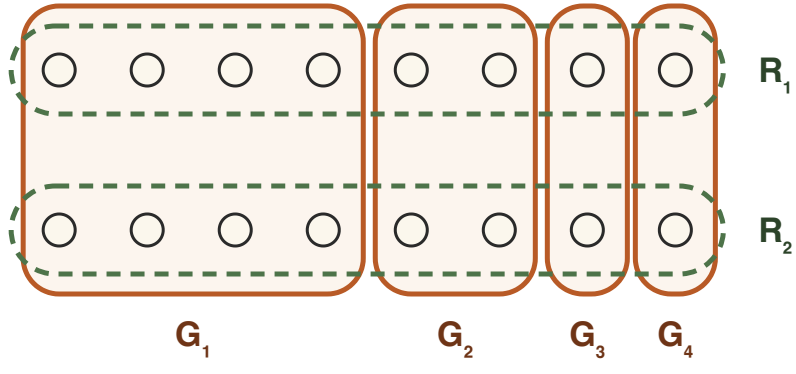
Order elements by when greedy covers them, last first. The k-th from last pays at most OPT/k, so:

$$\begin{aligned} \#\text{sets} &= \sum_{e} \text{price}(e) \;\le\; \sum_{k=1}^{n} \frac{\mathrm{OPT}}{k} \\ &= H_n \cdot \mathrm{OPT} \;\le\; (\ln n + 1)\,\mathrm{OPT}. \;\blacksquare \end{aligned}$$

CHARGING, STEP 3: SUM THE HARMONIC SERIES

- The last element covered is the expensive one ($k = 1$: price up to OPT); early elements are nearly free. The harmonic sum totals the damage.
- The same proof handles *weighted* set cover (pick the set minimizing cost per new element): factor H_n again.
- Applied to vertex cover it gives $O(\log n)$ — worse than our factor 2! General-purpose tools pay for their generality.

GREEDY REALLY CAN LOSE A LOG FACTOR



$n = 16$: OPT is the 2 rows R_1, R_2 (green, dashed). Greedy, with ties broken badly, picks the halving blocks G_1 (8), G_2 (4), G_3 (2), G_4 (2) — 4 sets.

Scale to 2×2^k with block sizes $2^k, 2^{k-1}, \ldots$ (perturb sizes to force the ties): greedy uses $\approx \log_2 n$ sets while $\mathrm{OPT} = 2$. The H_n analysis is essentially tight.

AND NOTHING BEATS GREEDY (PROBABLY)

Theorem For every $\varepsilon > 0$, a $(1 - \varepsilon) \ln n$-approximation for set cover would imply P = NP.

Dinur & Steurer 2014, completing a line started by Lund–Yannakakis 1994 and Feige 1998.

- So the most naive algorithm imaginable is *optimal to within the constant in front of* $\ln n$.

AND NOTHING BEATS GREEDY (PROBABLY)

- Moral of the week: approximability is a spectrum. VC and metric TSP sit at constant factor; set cover at $\Theta(\log n)$; general TSP at "nothing, ever". NP-completeness (W11) collapses all of these to "hard" — approximation reveals the finer structure.

PART 7: THE END OF THE VERTEX-COVER STORY

ONE PROBLEM, TWELVE WEEKS

Setting	Status	Tool	Week
Trees	poly, exact	DP on trees; Gallai identity	W3
Bipartite graphs	poly, exact	Kőnig: alternating paths, min cut, LP duality	W3, W8, W10
General graphs	NP-complete	reductions 3SAT $\longrightarrow$ IS $\longrightarrow$ VC	W11
General graphs	2-approximable	matching, LP rounding, primal–dual	W12